

Laboratorio 2

Introducción a lenguaje Ensamblador

Tiago Reino

email:tiago.reino@estudiantes.utec.edu.uy

Sofia Lapitz

email:sofia.lapitz@estudiantes.utec.edu.uy

David Santana

email:david.santana@estudiantes.utec.edu.uy

Link para acceder a los videos en Google Drive:

<https://drive.google.com/drive/folders/1zcI38ctbUfpSJ5X0M6irgypbLaV2Cd7T?usp=sharing>

Link para acceder al repositorio GitHub:

<https://github.com/tiagoreino/Laboratorio-2.git>

Resumen—En el presente informe se aborda la resolución de cinco problemas de diseño digital desarrollados en lenguaje ensamblador para el microcontrolador ATmega328P y el ATmega2560. Las aplicaciones comprenden el control de una matriz de LEDs, la síntesis de señales analógicas mediante un DAC R-2R con LUT, el control de un plotter electromecánico, la comunicación serie punto a punto (USART) y la automatización de un portón mediante una máquina de estados con interrupciones de seguridad. Con ello se consolida la integración de los periféricos internos del AVR, tales como temporizadores, comunicación UART, memoria, Stack Pointer e interrupciones por hardware.

Index Terms—Microcontrolador Lenguaje ensamblador ATmega328P ATmega2560 MAX7219 Stack Pointer Timers Interrupciones LUT USART UART DAC R-2R

I. INTRODUCCIÓN

Para la etapa de validación previa de los programas en ensamblador, se utilizaron entornos de simulación virtual antes del montaje físico. Se empleó PicSimLab para la verificación lógica de los periféricos internos en la mayoría de los desarrollos, y Proteus para la simulación de las señales analógicas mediante circuito R-2R.

En este contexto, la presente práctica de laboratorio propone el desarrollo e implementación en lenguaje ensamblador de cinco sistemas de complejidad creciente sobre el microcontrolador ATmega328P y el ATmega2560, abarcando desde el control de periféricos de visualización y conversión digital-analógica hasta el diseño de un sistema automatizado para una puerta de garaje mediante una máquina de estados finitos (FSM) gestionada por interrupciones. Asimismo, se aborda el control temporizado de un plotter industrial operado por PLC y la comunicación serie bidireccional (USART) entre dispositivos. A través de estos problemas se busca consolidar el uso práctico de los principales recursos de la arquitectura AVR: la gestión del Stack Pointer, los modos de direccionamiento de memoria, los temporizadores y contadores, las interrupciones externas, los puertos de entrada/salida y el uso del módulo de comunicación USART.

El presente informe se organiza de la siguiente manera: la Sección II detalla los objetivos generales y específicos perseguidos; la Sección V describe el diseño y la implementación adoptados para cada uno de los cuatro problemas propuestos; la Sección VI presenta lo obtenido junto con las principales dificultades encontradas durante el desarrollo; y finalmente, la Sección VII sintetiza los aprendizajes derivados de la experiencia.

II. OBJETIVOS

II-A. Objetivo General

En esta segunda instancia de laboratorio se busca profundizar en el uso del microcontrolador ATmega328P y la programación en lenguaje ensamblador, incorporando los diferentes periféricos de la arquitectura AVR. Se trabajará con el manejo del Stack Pointer y el direccionamiento de memoria, temporizadores y contadores, interrupciones, tablas de consulta mediante LUT (Look-Up Table) y comunicación serie utilizando USART o UART, con el objetivo de integrar estos recursos en aplicaciones prácticas.

II-B. Objetivos Específicos

- **Configurar** el periférico USART del microcontrolador ATmega328P para el envío y recepción de menús interactivos y datos en tiempo real.
- **Establecer** la comunicación serie punto a punto entre dos microcontroladores para el intercambio e interpretación de datos binarios.
- **Validar** el correcto funcionamiento de los programas en ensamblador mediante simulación previa y pruebas en hardware real.

III. MATERIALES UTILIZADOS

Se utilizaron los siguientes materiales:

- Microcontrolador ATmega2560.
- Microcontrolador ATmega328P.

- Protoboard.
- Fuente de alimentación.
- LEDs.
- Resistencias.
- Matriz de LEDs 8x8 MAX7219.
- Osciloscopio.
- Plotter.
- Pulsadores.
- Cables.

IV. MARCO TEÓRICO

El conversor 2R-R es un DAC (Digital-to-Analog Converter) Conversor Digital a Analógico que está construido únicamente con Resistencias. Esto quiere decir que tomaremos un valor digital de una cantidad X de bits y será traducido a una señal equivalente analógica. Por ejemplo, si tenemos un DAC de 3 bits alimentado con 5V, el 101 se traducirá a un valor aproximado de 3,125V.

El stack pointer (puntero de pila o registro SP) es un registro especial en el microcontrolador que guarda la dirección de memoria de la última posición utilizada en la pila (stack), un área de la memoria RAM reservada para almacenar datos de forma temporal. [1]. Este puntero funciona con la lógica LIFO: Last In, First Out (el último dato en entrar es el primero en salir) y permite guardar direcciones de memoria de forma temporal para poder hacer uso de interrupciones o llamadas a subrutinas (rcall) de forma que se registre la dirección de retorno.

Una interrupción en un microcontrolador es una señal de hardware o software que pausa de forma temporal el programa principal para que el procesador atienda un evento urgente. [2]. Normalmente primero se detiene la tarea que está haciendo en el bucle principal, se guarda la dirección actual (Stack Pointer) y los registros para no perder el hilo de trabajo, salta a una función especial llamada Rutina de Servicio de Interrupción (ISR) para resolver el evento y finalmente restaura los valores guardados y continúa con el programa principal exactamente donde se quedó.

Una LUT o Look-Up Table (Tabla de Búsqueda) en un microcontrolador es una estructura de datos que se utiliza para sustituir cálculos matemáticos complejos o lógica en tiempo de ejecución por una simple operación de acceso a la memoria. De esta forma se busca un tiempo de ejecución menor, a costa de ocupar mayor espacio. Normalmente se utiliza en la generación de señales: Almacenar los puntos de una onda senoidal, triangular o cuadrada para leerlos secuencialmente con un temporizador (Timer) y sacarlos por un DAC (Convertidor Digital a Analógico), o para funciones trigonométricas o logarítmicas. Ej: Calcular $\sin(x)$, $\cos(x)$, $\log(x)$.

El MAX7219 es un circuito integrado controlador de displays LED, capaz de manejar hasta 64 LEDs individuales organizados como una matriz de puntos de 8x8, o hasta ocho dígitos de 7 segmentos, mediante una interfaz serie de tres hilos: datos (DIN), reloj (CLK) y selección de chip (CS o Load). Internamente dispone de una memoria de video que almacena el estado de cada una de las 8 filas, además de

registros de configuración para el control del brillo, el límite de escaneo (cantidad de filas activas) y el modo de test de pantalla, lo que permite reducir significativamente la cantidad de pines del microcontrolador necesarios para controlar una gran cantidad de LEDs, delegando en el propio chip el multiplexado y refresco de la matriz [3].

Un USART (Universal Synchronous Asynchronous Receiver-Transmitter) es un periférico de hardware en un microcontrolador que permite la comunicación serial tanto de forma síncrona como asíncrona, mientras que un UART (Universal Asynchronous Receiver-Transmitter).

El Pen Plotter es una máquina la cual produce dibujos gráficos vectoriales. Los trazadores dibujan líneas de papel usando una lapicera. [4]

Un controlador lógico programable, más conocido como PLC, es una computadora utilizada en la ingeniería, para automatizar procesos. Diseñado para múltiples señales de entrada y de salida, rangos de temperatura ampliados, inmunidad al ruido eléctrico y resistencia a la vibración y al impacto. Los resultados de salida son producidos en respuesta a las condiciones de entrada dentro de un tiempo limitado, de lo contrario no produce el resultado deseado. [5]

Proteus es una aplicación para la ejecución de proyectos de construcción de equipos electrónicos en todas sus etapas: diseño del esquema electrónico, programación del software, construcción de la placa de circuito impreso, simulación de todo el conjunto, depuración de errores, documentación y construcción.

V. METODOLOGÍA

V-A. Problema A. Matriz de LEDs

Se desarrolló un sistema para controlar la matriz de LEDs, capaz de desplegar un mensaje desplazándose de derecha a izquierda de forma continua, y de permitir, mediante comunicación UART, la selección entre dicho mensaje y las figuras ya implementadas en la evaluación continua de la matriz.

Los mensajes elegidos fueron "MALLA" "MARCELOO EMPRENDE VUELO", cumpliendo con el requisito de tratarse de una frase con sentido completo de al menos 10 caracteres. Cada carácter del mensaje se representó mediante una fuente de mapa de bits almacenada como LUT en memoria de programa, donde cada columna de la matriz corresponde a un byte que indica el estado de los LEDs de esa columna.

V-A1. Comunicación con la matriz (MAX7219): La matriz se controla mediante el circuito integrado MAX7219, conectado al microcontrolador a través de tres líneas: CS (Chip Select/Latch, PB0), CLK (reloj, PB1) y DIN (entrada de datos, PB2). La comunicación se implementó mediante la subrutina SHIFT_OUT, que envía un byte bit a bit, comenzando por el más significativo, hacia DIN, sincronizado con pulsos de CLK, replicando en software el protocolo serie que exige el chip. Cada transacción completa (dirección de registro seguida del dato correspondiente) se realiza en SEND_MAX7219, que baja la línea CS antes de invocar dos veces a SHIFT_OUT y la sube al finalizar, momento en el cual el MAX7219 efectivamente aplica (latchea) el dato recibido.

Al inicio del programa, INIT_MAX7219 configura los registros internos del controlador: se desactiva la decodificación

por dígitos de 7 segmentos (para operar en modo matriz de puntos), se fija el nivel de brillo, se habilita el límite de escaneo en las 8 filas y se activa el modo de encendido, desactivando además el modo de test de pantalla.

V-A2. Modos de visualización: El sistema ofrece ocho modos, seleccionables mediante un carácter ('1' a '8') recibido por UART: los modos 1 y 2 despliegan los mensajes con desplazamiento continuo antes mencionados, mientras que los modos 3 a 8 muestran figuras estáticas (un asterisco, una cara "XD", una cara feliz, un guiño, una carita ÜwUz un corazón). Tanto los mensajes como las figuras se almacenan como LUTs en memoria FLASH mediante .db: cada carácter de un mensaje ocupa un bloque de bytes (una columna de la matriz por byte), mientras que las figuras estáticas ocupan directamente los 8 bytes correspondientes a sus 8 filas.

Para el desplazamiento del texto (DESLIZAR_TEXTO) se mantiene en SRAM un buffer de 8 bytes que representa el estado actual de la matriz. En cada iteración se lee un nuevo byte de la LUT del mensaje mediante LPM, se desplazan los 7 bytes existentes del buffer una posición hacia la izquierda y se inserta el nuevo byte en la última posición, mostrando el resultado en la matriz (MOSTRAR_BUFFER) y aplicando un retardo (DELAY) antes de repetir el ciclo. El fin del mensaje se identifica mediante un byte centinela (0xFE) al final de la LUT. Las figuras estáticas, en cambio, se cargan directamente desde FLASH a la matriz mediante MOSTRAR_ESTATICO, sin necesidad de manipular un buffer intermedio.

V-A3. Comunicación UART y control de velocidad: La recepción de datos por UART se implementó mediante interrupción (RXCIE0 habilitado) en lugar de sondeo continuo, lo cual permite que el cambio de modo sea inmediato: dentro de ISR_UART_RX, si el carácter recibido corresponde a un dígito de modo, se actualiza directamente la variable MODO_ACTUAL. Tanto DESLIZAR_TEXTO como DELAY comparan periódicamente este valor contra una copia local tomada al inicio de su ejecución, abandonando la operación en curso apenas se detecta un cambio, sin esperar a que finalice el ciclo completo en curso.

Adicionalmente, la misma interrupción de UART se utilizó para implementar un control de velocidad del desplazamiento: al recibir los caracteres '+' o '-', se incrementa o decrementa en 5 unidades, dentro de un rango acotado entre 5 y 80, la variable VELOCIDAD, almacenada en SRAM y consultada por DELAY en cada retardo entre columnas, permitiendo ajustar la velocidad de desplazamiento en tiempo real sin necesidad de reiniciar el programa.

Dado que ISR_UART_RX se ejecuta de forma asincrónica respecto al programa principal, pudiendo interrumpir, por ejemplo, la transmisión de un dato al MAX7219 (SHIFT_OUT) o la manipulación del buffer de desplazamiento, se decidió resguardar explícitamente en la pila el registro de estado (SREG) y los registros de trabajo empleados dentro de la interrupción (TEMP y R20), restaurándolos antes de retornar con reti, evitando así alterar el estado del programa interrumpido.

V-B. Problema B. 2R-R

Se desea implementar un convertor Digital-to-Analog (DAC) de tipo 2R-R, de forma que:

- **A:** Se emplee una LUT para el almacenamiento y posterior lectura de las señales predefinidas.
- **B:** Dicha señal debe traducirse mediante el convertor 2R-R, de forma que pueda visualizarse en un osciloscopio la onda generada por el microcontrolador, con frecuencia variable mediante la manipulación de un temporizador.
- **C:** La selección de la señal a mostrar debe realizarse a través de un menú desplegado por puerto Serial.

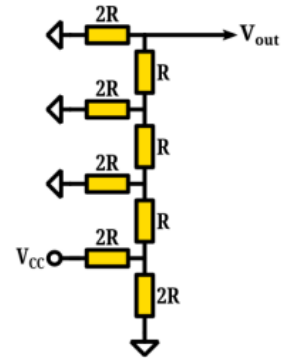


Figura 1. Circuito 2R-R Digital-to-Analog (DAC)

Las señales asignadas a este equipo fueron la número 2 y la número 4, correspondientes a una señal cuadrada y a una señal diente de sierra de pendiente negativa, respectivamente.

V-B1. Código: Al igual que en los problemas anteriores, se parte de la librería base para el ATmega328P. A diferencia de los códigos previos, este problema requiere de una interrupción por coincidencia de comparación del Timer1 (Compare Match A), por lo que se redirige el vector correspondiente con .org 0C1Aaddr hacia la rutina TIMER1_COMPA.

Se definen dos LUTs, una por señal (TABLA_7SEG para la señal cuadrada y TABLA_SENAL2 para la señal diente de sierra), ambas apuntando a la misma dirección de memoria SRAM (0x0256). Esto es posible porque solamente una de las dos señales se encuentra activa en cada momento, por lo que no existe riesgo de superposición de datos entre ambas tablas.

En start se inicializa el Stack Pointer y la comunicación UART, necesaria para desplegar el menú por puerto Serial. Luego se llama a la subrutina configurar, que configura como salidas los pines PC0-PC5 (bits 0-5 del DAC) y PB0-PB1 (bits 6-7 del DAC), además de PB5, utilizado como LED indicador visual de cada interrupción del Timer1. También se inicializan en 0 las salidas del DAC.

A continuación se llama a la subrutina MENU, la cual despliega por UART las opciones disponibles (1 - SEÑAL 1, 2 - SEÑAL 2) y espera, mediante getch, a que el usuario ingrese el carácter correspondiente. Según la opción seleccionada, se guarda en el registro r25 un valor (1 o 2) que identifica la señal activa, y se reinicia en 0 el registro r24, utilizado como índice de posición dentro de la LUT correspondiente.

Los pulsadores PD2 y PD3, utilizados para variar la frecuencia de la señal, se configuraron como entradas con resistencia de pull-down externa (a diferencia del resto de los problemas del laboratorio, en los que se emplearon pull-ups internas), de forma que su valor en reposo es 0 y pasa a 1 al ser presionados.

Antes de configurar el temporizador, se establece en `r17:r18` el valor inicial de OCR1A (15624), correspondiente a una interrupción cada 1 s con el microcontrolador funcionando a 16 MHz y un preescalador de 1024. Este valor se emplea luego en `CONFIG_TIMER1`, donde se configura el Timer1 en modo CTC (*Clear Timer on Compare Match*), se carga OCR1A con dicho valor, se habilita la interrupción por coincidencia de comparación A (OCIE1A) y se habilitan las interrupciones globales mediante `sei`.

El programa principal (`main_loop`) delega el control a `ESPERAR_PULSACION`, un bucle que evalúa constantemente el estado de PD2 y PD3. Si se detecta PD2 en alto, se llama a `DISMINUIR_TIEMPO`: tras aplicar un retardo antirrebote (`DELAY`) y confirmar que el pulsador continúa presionado, se divide entre 2 el valor de `r17:r18` mediante un desplazamiento a la derecha encadenado (`lshr r17 / ror r18`), reduciendo así el valor de OCR1A y, en consecuencia, aumentando la frecuencia de la señal generada. De forma análoga, si se detecta PD3 en alto, `AUMENTAR_TIEMPO` multiplica por 2 el mismo par de registros (`lsl r18 / rol r17`), aumentando el valor de OCR1A y disminuyendo la frecuencia. En ambos casos, el nuevo valor se escribe directamente en OCR1A y el programa espera a que el pulsador se suelte (`ESPERAR_SUELTA`) antes de volver a evaluar entradas, evitando que una única pulsación se interprete como múltiples cambios de frecuencia consecutivos.

Toda la generación de la señal ocurre dentro de la rutina de interrupción `TIMER1_COMP`, ejecutada cada vez que el Timer1 alcanza el valor de OCR1A. Al ingresar a la interrupción se guardan en la pila el registro de estado (SREG) y todos los registros utilizados dentro de la ISR, evitando así alterar el estado del programa principal. Luego se conmuta el pin PB5 (`sbi PINB, PB5`), utilizado únicamente como indicador visual de que la interrupción se está ejecutando. A continuación se evalúa el valor de `r25` para determinar qué señal se encuentra seleccionada: si corresponde a la señal 1, se llama a `get_7seg_code`; si corresponde a la señal 2, se llama a `get_senal2_code`. Ambas subrutinas toman como índice el valor actual de `r24` y devuelven en `r20` la muestra correspondiente de la LUT.

El valor obtenido se envía al conversor 2R-R mediante la subrutina `SALIDA_DAC`, la cual separa el byte de 8 bits entre los dos puertos utilizados: los 6 bits menos significativos se escriben directamente en PORTC (PC0-PC5), mientras que los 2 bits más significativos se desplazan seis posiciones a la derecha y se combinan, mediante una máscara, con el valor actual de PORTB, de forma de escribir únicamente PB0 y PB1 sin alterar el estado de PB5. Finalmente, se incrementa `r24` para avanzar a la siguiente muestra en la próxima interrupción; al tratarse de un registro de 8 bits, el índice se reinicia automáticamente a 0 al superar el valor 255, logrando que la señal se repita de forma periódica sin necesidad de una comprobación explícita de desborde.

V-C. Problema C. Plotter

En el caso particular del Laboratorio de Mecatrónica, se cuenta con un plotter monocromático el cual es operado por un Controlador Lógico Programable (PLC) que será utilizado para llevar a cabo la Parte C del segundo laboratorio de la unidad curricular. Ver Figura 2.

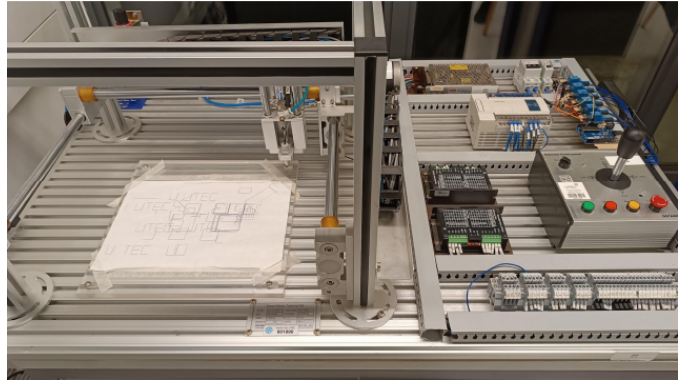


Figura 2. Plotter del Laboratorio de Mecatrónica

La ploteadora cuenta con un microcontrolador ATmega328p conectado mediante el uso de relés electromecánicos para enviar señales al PLC siguiendo la tabla I

Cuadro I
ASIGNACIÓN DE CONEXIONES A PINES DIGITALES

Pin Digital	Conexión
D2	Bajar solenoide X0
D3	Subir solenoide X1
D4	Movimiento hacia abajo X5
D5	Movimiento hacia arriba X6
D6	Movimiento hacia la izquierda X7
D7	Movimiento hacia la derecha X10

Utilizando esa tabla como referencia, se desea programar un ATmega328P en Ensamblador para controlar un plotter de dibujo mediante comunicación serial, permitiendo al usuario seleccionar y dibujar diversas figuras. El sistema debe ser capaz de:

- Dibujar las figuras predefinidas: **1** para triángulo, **2** para círculo, **3** para pentagrama, **4** para una figura libre (diseñada por ustedes), **P** para dibujar el pokemon asignado y **T** para todas las figuras.
- Ejecutar las secuencias de dibujo considerando los tiempos de conmutación de los relés o MOSFET, garantizando precisión en el movimiento del plotter.
- La edición personalizada de los dibujos por parte del usuario.

Esto teniendo en cuenta que nuestra figura personal fue una casa simple con una ventana y una puerta. Mientras que nuestro pokemon sorteado fue Grimer.

V-C1. Código Prueba: Dado a que no se tenían las especificaciones exactas de la ploteadora, como las dimensiones, el tamaño de los pasos según su velocidad, o si dicho tamaño era igual entre todas sus dimensiones, se decidió elaborar un

primer código de prueba (`Prueba.asm`) orientado exclusivamente al control manual del plotter. Este código reutiliza la librería base para ATmega328P, la configuración y cálculo de baudios por segundo (*bps*), la subrutina `configurar` (que habilita PD2-PD7 como salidas) y la subrutina `initUART`.

El control manual se implementó mediante seis macros (derecha, izquierda, arriba, abajo, subir, bajar), cada una asociada a un bit distinto de PORTD según la tabla I. Cada macro activa el bit correspondiente, invoca la subrutina de *delay* y luego apaga la salida, replicando el pulso necesario para accionar el relé/MOSFET asociado a ese movimiento. Estos comandos se asignaron a las teclas **W**, **A**, **S**, **D** (movimiento en los cuatro ejes) y **Q**, **E** (subir y bajar el marcador), recibidas mediante la subrutina `getc` a través de UART, siguiendo un esquema similar al de un control tipo videojuego. Se agregó además la tecla **P**, que dispara una secuencia fija de prueba (bajar, subir, derecha, izquierda, arriba, abajo), utilizada para verificar de forma rápida que los seis relés respondieran correctamente antes de avanzar con pruebas más finas; Además, este último comando permite comprobar si los movimientos con sentidos opuestos tienen la misma velocidad y recorrido, pues si después de hacer los seis posibles movimientos vuelve a la misma posición, implica que los movimientos con sentidos contrarios tienen igual recorrido.

El aspecto más relevante de este código de prueba fue la subrutina `delay`, construida a partir de `delay_1ms` (un retardo calibrado en aproximadamente 1 ms) invocada repetidamente según el valor de `DELAY_MS`. Variando este parámetro y midiendo el desplazamiento físico resultante en la hoja, se pudo determinar de forma empírica la relación entre tiempo de activación y distancia recorrida por el cabezal, dato que la ploteadora no especificaba y que resultó imprescindible para diseñar con precisión el código definitivo.

V-C2. Código Definitivo: El código definitivo reutiliza la base del código de prueba (macros de movimiento, `initUART`, `getc`, `delay`) y conserva el control manual por teclado, pero incorpora la capacidad de dibujar las figuras predefinidas de forma automática. Para esto se agregaron los comandos **C** (círculo), **T** (triángulo), **G** (Grimer), **P** (pentagrama), **V** (casita, la figura libre del equipo) y **F** (secuencia con las cinco figuras).

Cada figura se representa como una lista ordenada de pasos, almacenada en la memoria FLASH mediante `.db`, donde cada byte codifica un movimiento elemental según una tabla de equivalencias definida con directivas `.equ` (`MOV_DERECHA = 1`, `MOV_IZQUIERDA = 2`, ..., hasta dos códigos adicionales para movimientos diagonales, y `FIN = 0` como marca de cierre del arreglo). Esta codificación numérica, similar en concepto a una LUT, permite describir cualquier dibujo como una simple sucesión de números, lo que facilita enormemente tanto la depuración como la edición manual de una figura sin necesidad de recordar instrucciones de ensamblador.

La subrutina `siguiente_numero` recorre este arreglo utilizando el puntero `Z` y la instrucción `lpm`, leyendo un byte a la vez hasta encontrar el valor `FIN`. Cada byte leído se delega a `interpretar_numero`, que mediante una cadena de comparaciones (`cmp/breq`) traduce el código numérico a la macro de movimiento correspondiente. Los dos códigos

adicionales (diagonales) se resolvieron activando simultáneamente dos bits de PORTD en una única salida (por ejemplo, derecha + arriba), lo que permitió trazar líneas oblicuas sin necesidad de definir una macro exclusiva para cada diagonal.

Para adaptar el tamaño de cada figura sin modificar su secuencia de pasos, se definió un *delay* independiente por figura (`DELAY_CIRCULO`, `DELAY_TRIANGULO`, `DELAY_GRIMER`, `DELAY_PENTAGRAMA`, `DELAY_CASITA`). De esta forma, escalar cualquier dibujo se reduce a modificar una única constante, en lugar de tener que reescribir la secuencia de movimientos.

Finalmente, el comando **F** dispara la subrutina `ejecutar_secuencia`, que dibuja las cinco figuras de forma consecutiva en distintas posiciones de la hoja. Entre figura y figura, el cabezal se reposiciona mediante desplazamientos manuales (`int_derecha`, `int_abajo`, etc.) antes de bajar el marcador, subiéndolo nuevamente al finalizar cada dibujo, de forma que las cinco figuras queden distribuidas sobre la hoja sin superponerse. Dado que el sistema no lleva un registro interno de la posición absoluta del cabezal, es indispensable pulsar el botón físico de RESET del plotter antes de iniciar la secuencia, para garantizar que el punto de partida coincida siempre con la posición de referencia (esquina superior derecha) usada al calcular los desplazamientos entre figuras.

V-D. Problema D. Comunicación USART - UART

Se desea diseñar e implementar una comunicación entre dos microcontroladores ATmega328P mediante USART-UART. Para esto se requiere que el primer microcontrolador (Emisor) contenga 3 pulsadores conectados, los cuales permitirán seleccionar un número binario entre el 0 y el siete. Dicho valor debe ser transmitido mediante USART hacia el segundo microcontrolador (Receptor). El microcontrolador Receptor debe ser capaz de recibir la el número enviado por el Emisor, y de acuerdo al número recibido activar una salida correspondiente al valor numérico recibido. Para la visualización de la activación de las salidas del Receptor, será necesario el uso de LEDs. De esta forma, si el usuario ingresa el número 011 (equivalente al 3 decimal) entonces se deberá encender el 3er LED conectado al Receptor.

Para el desarrollo de la propuesta se empezó por entender que la aparición de un segundo microcontrolador implica el desarrollo de dos códigos independientes. Teniendo eso claro se empezó por el desarrollo del Emisor.

V-D1. Lógica Emisor: Primeramente se inicializa el programa como siempre con librería base de `.include "m328pdef.inc"`. Se calcula los baudios por segundo (*bps*) con la línea `.equ bps = (f_cpu/16/ baud) - 1`.

El bloque de `inicio` inicializa el *Stack Pointer* para futuras llamadas a subrutinas (`rcall`). Se inicializa la comunicación UART con `rcall initUART` que activa los puertos RX y TX.

Las entradas del sistema se distribuyen entre los tres primeros puertos de B, configurados como entradas con resistencias de pull-up internas habilitadas:

- **PB0-PB2:** Selecciona el número que será enviado por el microcontrolador Emisor

Dentro del bucle principal (`main_loop`) se reinicia el valor del registro 16 (`r16`). Después comprobamos si los valores de los pines de B (`B0-B2`) son 0, si lo son, entonces se aplica `ori` a `r16` y después se llama a la subrutina `putnum` que envía el valor guardado en `r16` al microcontrolador Receptor.

V-D2. Lógica Receptor: Ambos programas son bastante parecidos en muchos aspectos (inicialización de UART, Stack Pointer, librería base `m328pdef.inc`, cálculo bps). La diferencia en la lógica de este segundo microcontrolador radica en que ahora el bucle principal (`main_loop`) inicia con una llamada a la subrutina (`getnum`) la cual toma los valores que ingresan por UART y los guarda en el registro 16 del microcontrolador Receptor.

Seguido a esto, coincidimos en equipo que la mejor forma de seleccionar la salida de cada uno de los números que podían ser ingresados era mediante una Look-Up-Table (*LUT*). Para el uso de la misma se llama a `get_u`.

Antes de seguir con la lógica del Receptor hay que hacer una aclaración respecto con los puertos; y es que en un principio se utilizaron todos los puertos B (**PB0-PB7**) El problema con esto es que los puertos **PB6-PB7** en las *ATmega328p* no existen. Esta confusión fue a causa de la costumbre de trabajar con un microcontrolador *ATmega2560* que sí posee estos puertos. Es por esto que al final se optó por la siguiente disposición de puertos para los LEDs de salida del microcontrolador Receptor:

- **PB0-PB5:** Muestra su activación para los números recibidos entre el 0-5 en decimal
- **PC0-PC1:** Muestra su activación para los números recibidos entre el 6-7 en decimal

Una vez obtiene el valor guardado de la tabla gracias a `get_u`, compara el valor encontrado para saber si debe activar el puerto B o puerto C con las líneas `cpi r22, 6` y `brsh salida_PORTC`. Dependiendo de esta comparación, activa un puerto y apaga el no seleccionado.

V-E. Problema E. Automatización de una puerta de garaje

Se deberá programar en lenguaje ensamblador un microcontrolador *ATmega328P* para controlar el funcionamiento de una puerta de garaje. El sistema contará con dos pulsadores, control remoto, uno para solicitar la apertura y otro para solicitar el cierre. Además, dispondrá de dos sensores de posición: uno para indicar que la puerta se encuentra completamente abierta y otro para indicar que está completamente cerrada. El movimiento de la puerta será controlado mediante un motor, utilizando dos salidas del microcontrolador para indicar las órdenes de apertura y cierre. Durante el movimiento de la puerta deberá activarse una alarma sonora, con el objetivo de advertir a las personas que un vehículo está ingresando o saliendo del garaje. Por razones de seguridad, se incorporará un sensor de obstáculos conectado a una entrada de interrupción (emplear una interrupción diferente a `INT0` o `INT1`)¹. Si se

detecta un obstáculo mientras la puerta está en movimiento, la interrupción deberá detener inmediatamente el motor y desactivar la alarma hasta volver a presionar los pulsadores del control remoto. Además, el microcontrolador deberá utilizar la comunicación `USART` para informar el estado del sistema. Se deberán transmitir mensajes que permitan identificar, como mínimo, los siguientes eventos:

- Puerta abriendo.
- Puerta abierta.
- Puerta cerrando.
- Puerta cerrada.
- Obstáculo detectado.
- Movimiento detenido por seguridad.

Debido a la naturaleza del problema, que incluye interrupciones, entradas y salidas, actuadores y estados definidos; se decide desarrollar un diagrama de máquina de estados para la resolución de este problema, el mismo fue desarrollado gracias a la herramienta web `draw.io`. Ver figura 3

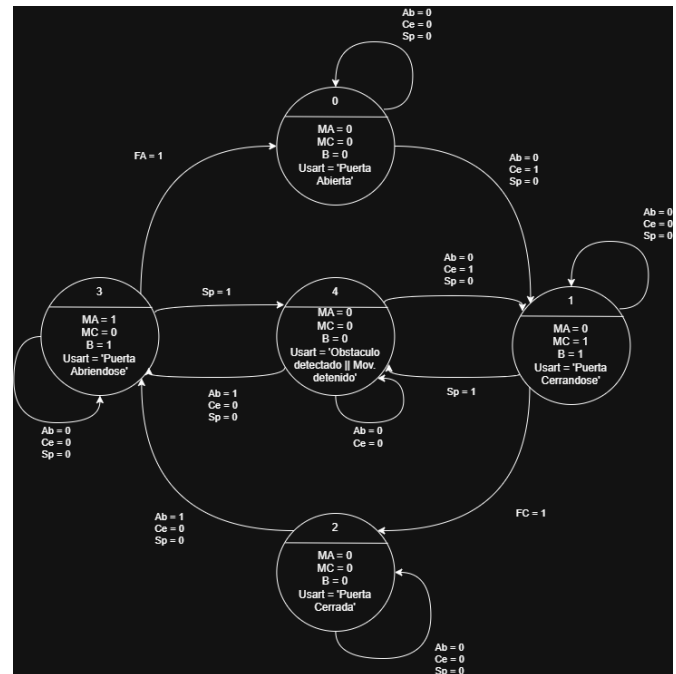


Figura 3. Máquina de Estados para Problema E: Garaje Automatizado

La Máquina de Estados Finita (FSM) desarrollada para la resolución de este problema cuenta con 5 estados, 5 entradas y 4 salidas (contando el mensaje mostrado por UART). Siguiendo las tablas II, III, IV.

Cuadro II
ESTADOS DE LA FSM

Estados	
0	Puerta Abierta
1	Puerta Cerrándose
2	Puerta Cerrada
3	Puerta Abriéndose
4	Obstáculo Detectado

¹Debido a los materiales que se disponían al hacer la práctica se utilizó un *ATmega2560*, el cual cuenta con 8 interrupciones del tipo `INT`. Aunque la consigna solo exige evitar `INT0` e `INT1`, se optó por no utilizar ninguna interrupción del tipo `INTn` (incluyendo `INT2-INT7`) y en su lugar implementar la detección mediante `Pin Change Interrupt (PCINT0)`, lo cual permite usar cualquier pin del puerto B sin depender de los pines fijos asignados a las interrupciones externas clásicas.

Cuadro III
 ENTRADAS DE LA FSM

Entradas	
Ab	Solicitar Apertura
Ce	Solicitar Clausura
Sp	Obstáculo Detectado
FA	Final Carrera Apertura
FC	Final Carrera Clausura

 Cuadro IV
 SALIDAS DE LA FSM

Salidas	
MA	Motor Apertura
B	Buzzer de Alerta
MC	Motor Clausura
USART	Mensaje Mostrado

V-E1. Flujo FSM: Inicialmente siempre empieza en el estado 0, donde sus salidas binarias son 0 y el mensaje mostrado es *Puerta Abierta*. La única entrada que cambia su estado es Ce (Representando la solicitud de clausura), una vez se activa, la máquina pasa al estado 1. En el estado 1 las salidas binarias MC (Representando movimiento de motor para clausura) y B tabulean a 1 y el mensaje mostrado cambia a *Puerta Cerrándose*. Una vez el FC cambie a 1, representado la activación del final de carrera, la máquina cambia al estado 2. En el estado 2 las salidas binarias vuelven a ser 0, mientras que el mensaje mostrado es *Puerta Cerrada*. Si se activa Ab (Representando la solicitud de apertura), la máquina pasa al estado 3, donde las salidas binarias MA (Representando movimiento de motor para apertura) y B se activan y el mensaje mostrado es *Puerta Abriéndose*. Una vez el FA cambie a 1, representado la activación del final de carrera, la máquina vuelve al estado 0. La máquina cambia al estado 4 cuando en el estado 1 o 3 se activa la entrada Sp, representando que mientras se movía la puerta, el sensor de movimiento detectó un obstáculo, en este caso, se desactivan todas las salidas binarias y se envía el mensaje *Obstáculo detectado, Movimiento detenido*. A partir del estado 4 se optó por que la máquina pudiera trasladarse al estado 1 o 3 dependiendo de la entrada seleccionada por el usuario.

V-E2. Código FSM: Teniendo como referencia la FSM elaborada, desarrollar el código fue mucho más fácil.

Primeramente se incluye la librería base para atmega2560, después se establecieron los punteros, tanto el principal `.org 0x0000` como el puntero de la interrupción seleccionada `.org 0x0012` (interrupción PCINT0). Cambiamos el nombre de los registros, entradas y salidas por comodidad, siguiendo como referencia el FSM elaborado.

En `start:` inicializamos *Stack Pointer*, configuramos *UART*, activamos entradas con Pull-Ups internas, activamos salidas, inicializamos flag (en este caso solamente el bit 0 de r18 para la entrada a parada de emergencia), habilitamos interrupción *PCINT* y habilitamos las interrupciones globales con `sei`.

En `main_loop` se encuentran todos los estados separados en bloques, cada bloque activa y desactiva cada salida e imprime el mensaje seleccionado dependiendo el estado. Por diseño se agregó un segundo bloque en cada estado para

quedarse comprobando la entrada para el siguiente estado, sin tener que imprimir el mensaje por cada iteración, de esta forma no se acumulan mensajes en terminal. Antes de cambiar de estado borramos la flag (Bit 0 r18) sin afectar las demás por si se quiere expandir el código con otras flags. En los estados de puerta abriéndose y puerta cerrándose se comprueba el estado de la flag, si esta se activa entonces quiere decir que debemos parar el proceso entrando en el estado 4 porque se ha detectado un obstáculo.

Para poder imprimir los mensajes en terminal se decidió hacer dos cosas. La primera es que guardamos una serie de bytes en la memoria FLASH de tal forma que podamos acceder a la misma a través de una llamada (similar a una LUT pero en vez de guardar un valor se guardan una serie de números) esto es posible con la instrucción `.db`, estos tienen el formato de "mensaje- nueva línea (13, 10) + fin de línea (0). Lo segundo que se hizo fue una subrutina que lea esta serie de bytes guardados en los registros Z carácter por carácter `print_string`.

VI. RESULTADOS

VI-A. Problema A. Matriz de LEDs

El sistema logró desplegar correctamente ambos mensajes con desplazamiento ("MALLLAz "MARCELOOO EMPRENDE VUELO"), así como las seis figuras estáticas predefinidas, todas seleccionables en cualquier momento mediante un carácter recibido por UART. Al implementarse la recepción por interrupción en lugar de sondeo continuo, el cambio de modo resultó inmediato incluso durante un desplazamiento de texto en curso, sin necesidad de esperar a que finalizara el ciclo de columnas activo.

En la primera instancia de presentación, el sistema no contaba con control de velocidad del desplazamiento, requisito contemplado en la consigna original. El docente señaló esta ausencia, pero permitió al equipo reagendar la presentación para incorporarla. Esto llevó a implementar el control de velocidad mediante los caracteres '+' y '-' enviados por UART, modificando en tiempo real la variable `VELOCIDAD` utilizada por la rutina de retardo entre columnas, dentro de un rango acotado (entre 5 y 80) que evita tanto un desplazamiento excesivamente rápido como uno demasiado lento. Tras esta incorporación, el sistema fue presentado nuevamente y aprobado en su totalidad.

VI-B. Problema B. 2R-R

El sistema logró generar correctamente ambas señales asignadas (cuadrada y diente de sierra con pendiente negativa), verificando su forma de onda en el osciloscopio conectado a la salida del conversor 2R-R. El menú desplegado por puerto Serial permitió seleccionar entre ambas señales sin inconvenientes, y los pulsadores de variación de frecuencia respondieron correctamente, permitiendo aumentar o disminuir la frecuencia de la señal generada mediante la modificación en tiempo real del registro `OCR1A`.

Cabe destacar una limitación identificada durante el desarrollo: las subrutinas `get_7seg_code` y `get_senal2_code`, invocadas en cada ejecución de

la rutina de interrupción del Timer1, reconstruyen la totalidad de la LUT correspondiente (256 muestras) en cada llamado, en lugar de limitarse a leer el valor ya almacenado en la posición indicada. Si bien esto no afectó el funcionamiento observado durante las pruebas realizadas, al tratarse de una operación que se ejecuta dentro de la propia interrupción, incrementa innecesariamente su tiempo de ejecución, lo cual podría volverse un problema ante frecuencias de muestreo más elevadas que las utilizadas en esta práctica, donde el tiempo de la ISR eventualmente podría superar al período configurado en OCR1A. La solución consistiría en guardar la LUT en memoria una única vez, al momento de seleccionar la señal desde el menú, dejando que la rutina de interrupción se limite exclusivamente a leer el valor correspondiente en cada muestra.

VI-C. Problema C. Plotter

El código desarrollado logró cumplir con todas las pautas establecidas: la ploteadora dibujó correctamente tanto la figura propia del equipo como el pokemon asignado. Además, los comandos agregados para el movimiento manual permitieron un control más completo del que exigía originalmente la consigna. Sin embargo, al momento de la presentación, el profesor señaló que algunas figuras no se dibujaban con precisión. Gracias a que el sistema desarrollado se basa en pasos tabulados mediante equivalencias numéricas, de configuración simple y directa, fue posible ajustar dichas figuras en el momento, hasta lograr un resultado que conformó al profesor. En consecuencia, esta práctica se considera un rotundo éxito.

VI-D. Problema D. Comunicación USART - UART

Si bien el error de la selección de puertos para el Receptor fue un error a solucionar, este fue rápidamente detectado y corregido. Al final el sistema logró de forma satisfactoria obtener un número de tres bits ingresado por el usuario, transferir esa información a través de comunicación UART de un microcontrolador a un segundo microcontrolador que recibió esa información, la interpretó de forma correcta y utilizó una LUT para clasificar que salida debía ser encendida. Este sistema opera a tiempo real y no presenta delay, resultando muy eficiente. Gracias a esto consideramos que la posibilidad de la comunicación UART es una alternativa muy útil a la hora de transmitir información compleja como la secuencia de datos.

VI-E. Problema E. Automatización de una puerta de garaje

El prototipo desarrollado logró cumplir con la consigna y demostró el correcto funcionamiento de la FSM elaborada para el desarrollo del problema. Las entradas respondían correctamente y las salidas también. Se logró implementar el uso de comunicación UART para informar el estado actual de la puerta. Se logró incorporar la interrupción deseada para la simulación de la detección de un objeto en los estados 1 y 3 como se preveía. Sin embargo, nuestro profesor encontró una falla en el diseño planteado, y es que se podía accionar el estado de abrir y cerrar con el botón de detección de objeto

pulsado. Aunque no se decía explícitamente en la consigna y el sistema en sí cumplía con lo propuesto, es verdad que no se podría considerar una representación fiel de una puerta real, pues eso supondría que la puerta podría abrirse o cerrarse con una persona o un objeto obstaculizando el recorrido de la puerta. La solución consistiría en verificar el nivel actual del pin PBO (`sbis PINB, PBO`) antes de permitir la transición a puertaAbriéndose o puertaCerrándose desde el estado de obstáculo, en lugar de limpiar la bandera incondicionalmente.

En términos generales, y una vez resueltas las dificultades mencionadas, los cinco sistemas desarrollados funcionaron correctamente de acuerdo con lo especificado en cada consigna. El código fuente completo, las simulaciones y los videos demostrativos de cada uno de los problemas se encuentran disponibles en el repositorio y el material audiovisual referenciados en la Sección VIII.

VII. CONCLUSIONES

El desarrollo de los cinco problemas propuestos en este segundo laboratorio permitió profundizar en el uso práctico de los principales periféricos de la arquitectura AVR más allá de lo abordado en la instancia anterior, incorporando de forma transversal el manejo del Stack Pointer para el direccionamiento de memoria y las llamadas a subrutinas, el uso de Look-Up Tables como estrategia recurrente para sustituir cálculos en tiempo de ejecución, ya sea para representar caracteres en una matriz de LEDs, muestras de una señal analógica, secuencias de movimiento de un plotter o valores de decodificación en una comunicación serie, la temporización mediante temporizadores por hardware en modo CTC, y las interrupciones tanto externas (Pin Change) como asociadas a la recepción de datos por USART.

En particular, la incorporación de comunicación serie en la mayoría de los problemas evidenció tanto sus ventajas, permitiendo interfaces de usuario simples e interactivas sin hardware adicional, como la importancia de resguardar adecuadamente el contexto de ejecución (registros y SREG) al atender interrupciones, dado que una recepción asíncrona puede ocurrir en cualquier punto del programa principal.

Asimismo, la experiencia reforzó la relevancia de validar mediante simulación (PicSimLab y Proteus) antes de pasar a la implementación física, así como la importancia de la retroalimentación docente durante el proceso de desarrollo, que en más de una ocasión permitió identificar tanto errores de diseño no evidentes en la etapa de simulación (como en el caso de la puerta de garaje) como requisitos pendientes de implementar (como el control de velocidad del Problema A), fortaleciendo el resultado final de cada sistema.

VIII. MATERIAL COMPLEMENTARIO

El código fuente completo de los cinco problemas desarrollados, junto con su historial de versiones, se encuentra disponible en el siguiente repositorio:

- **Repositorio GitHub:** <https://github.com/tiagoreino/Laboratorio-2.git>

Los videos demostrativos del funcionamiento de cada sistema en hardware físico, se encuentran disponibles en:

- **Videos (Google Drive):** <https://drive.google.com/drive/folders/1zcI38ctbUfpSJsX0M6irgypbLaV2Cd7T?usp=sharing>

REFERENCIAS

- [1] joseyli, *Puntero de Pila en Microcontroladores*, [En línea; consultado el 27-septiembre-2026], 2014. dirección: <https://www.scribd.com/document/246538108/La-Pila-Stack>
- [2] Stuart Ball, *Interrupciones del microcontrolador*, [En línea; consultado el 27-septiembre-2026], 2022. dirección: <https://circuitcellar.com/research-design-hub/basics-of-design/microcontroller-interrupts/>
- [3] MAXIM [Maxim Integrated Products], *MAX7219 (Serially Interfaced, 8-Digit LED Display Drivers)*, [En línea; consultado el 27-septiembre-2026], 2003. dirección: <https://www.alldatasheet.com/datasheet-pdf/view/73745/MAXIM/MAX7219.html>
- [4] Wikipedia contributors, *Plotter – Wikipedia, The Free Encyclopedia*, [En línea; consultado el 26-septiembre-2026], 2026. dirección: <https://en.wikipedia.org/wiki/Plotter>
- [5] Colaboradores de Wikipedia, *Controlador lógico programable – Wikipedia, la enciclopedia libre*, [En línea; consultado el 26-septiembre-2026], 2026. dirección: https://es.wikipedia.org/wiki/Controlador_l%C3%B3gico_programable